

1. Modelado del problema

El problema a abordar es un problema de control de stock, donde consideramos una empresa que vende varios productos, cada uno con su respectiva demanda. Estos productos son comprados todos al mismo proveedor y son enviados en barco en uno o más containers en cada período. Existe un costo de almacenamiento por mantener productos en stock, un costo implícito por dejar demanda insatisfecha al no tener stock del producto demandado (ventas que se pierden), y un costo por hacer un pedido con capacidad ociosa en el container.

Modelado de problema de control de stock con extensión multiproducto y capacidad de pedido compartida:

- Variables de estado

$$s_i = \{0, 1, \dots, N_i\}$$

$$S = [s_1, s_2, \dots, s_q]$$

S es un vector que representa la cantidad de stock que hay de cada uno de los productos i , ($i = 1, \dots, Q$), donde Q es la cantidad de productos diferentes que puede haber en stock. N_i es la capacidad del depósito para almacenar el producto i .

- Espacio de acciones

$$a_i = \{0, MOQ, MOQ + 1, \dots, MaxOrder\}$$

$$A = [a_1, a_2, \dots, a_q]$$

A es un vector que representa la cantidad pedida para reponer stock de cada uno de los productos i , ($i = 1, \dots, Q$), donde Q es la cantidad de productos diferentes que puede haber en stock. MOQ es una constante, igual para todos los productos, y representa la mínima cantidad que se debe ordenar del producto en caso de que este forme parte del pedido. $MaxOrder$ es la cantidad máxima que puede ordenarse de cada producto.

- Dinámica del sistema

La transición de s_i a s_i' se modela de la siguiente forma:

$$s_i' = \max\{0, s_i + a_i - d_i\}, \text{ para todo } i, i = 1, \dots, Q$$

donde:

s_i : stock anterior

a_i : ingreso de stock del periodo

d_i : demanda del periodo

La probabilidad de transición de s a s' , dada la acción a es:

$$P(s'|S = s, A = a) = P(D = s + a - s') \text{ si } s' \neq 0, \text{ si no } P(D \geq s + a)$$

- Función de recompensa

$$r_a = \sum_{i=1}^Q P_i * UV_{t,i} - ci_i * S_{t+1,i} - cu_i * a_i - cvp_i * VP_i - \text{Ceil}((\sum_{i=1}^Q a_i * v_i)/C) * cc$$

donde:

P_i : precio unitario del producto i

UV_i : unidades vendidas del producto i

ci_i : costo de almacenar inventario del producto i

cu_i : costo unitario de comprar el producto i

a_i : cantidad ordenada del producto i

cvp_i : costo por venta pérdida del producto i

VP_i : ventas perdidas del producto i $(D_i - UV_i)$ $\max(D_i - UV_i, 0)$

C : capacidad del container

v_i : volumen que ocupa el producto i

cc : penalidad por pedir un container con capacidad ociosa Por la expresión de recompensa, cc parece ser el costo de un container.

El modelo incluye incertidumbre a partir de la función de demanda, para la cual se asume una distribución normal truncada con distintos parámetros para cada uno de los productos. Por otro lado, la función de recompensa a optimizar incluye varios aspectos a considerar del problema, como una penalidad por tener capacidad ociosa en un container, el costo de dejar demanda insatisfecha y el costo de almacenar productos en stock. La dinámica temporal del problema está dada por la transición entre estados, donde el stock que sobró del periodo anterior afecta directamente al stock disponible en el periodo actual.

En este modelado se incluyen aspectos importantes del problema real, como es el hecho de que los productos comparten el envío en un mismo container, y la necesidad de que, de ser posible, el o los containers que se pidan no tengan capacidad ociosa. También se incluye la restricción real del MOQ , dada por el proveedor, y la estimación de demanda. Sin embargo, decidimos hacer algunas simplificaciones para la implementación numérica, como por ejemplo:

- Considerar únicamente 2 o 3 productos.

- Considerar una capacidad máxima de almacenamiento individual (N) y un máximo de unidades a ordenar ($MaxOrder$ individual para cada producto, cuando en la realidad esta capacidad es compartida ya que todos los productos comparten un mismo depósito.
- Asumir una distribución de demanda simplificada independiente del periodo t , en lugar de un forecast real que si depende de, por ejemplo, la demanda en el período anterior.
- Discretizar la demanda en 5 bins, cada uno con 0.2 de probabilidad de ocurrencia, a fin de limitar la cantidad de transiciones que cada demanda podía tener.
- Limitar tanto N como $MaxOrder$ a valores bajos para poder hacer los cálculos exactos. $N = 10$, $MaxOrder = 5$.
- Omitir la posibilidad de ampliar la capacidad de almacenamiento, en el problema real siempre es posible alquilar otro depósito o expandir el almacenamiento de otras formas.

Buenas simplificaciones!

2. Implementación numérica en Python

[Link Colab](#)

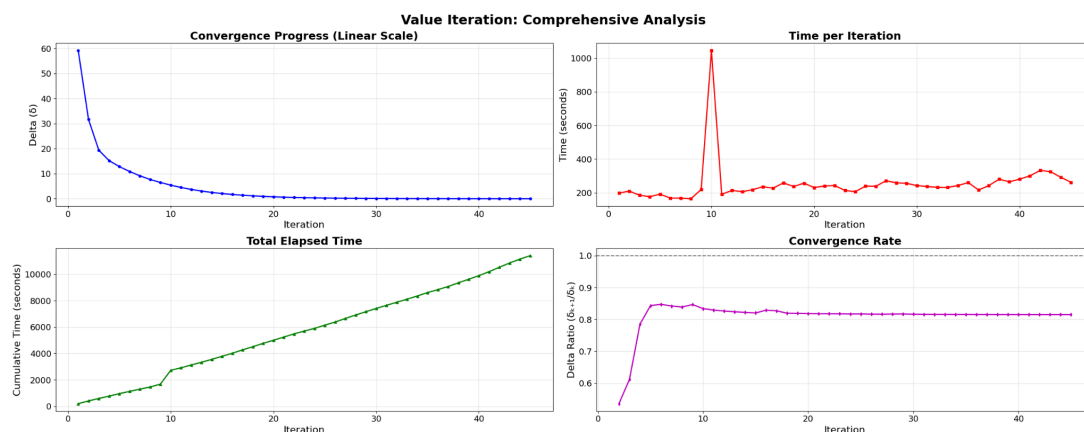
3. Modelado del problema

En el escenario de 3 productos el más rápido de todos fue Q Learning, con un tiempo de ejecución de 4 minutos, seguido por Policy Iteration (PI) que demoró 44 minutos. El algoritmo que más tardó en encontrar la solución fue Value Iteration (VI), que demoró 190 minutos.

En esa comparación, llevan en cuenta si Q-learning convergió a la política óptima?

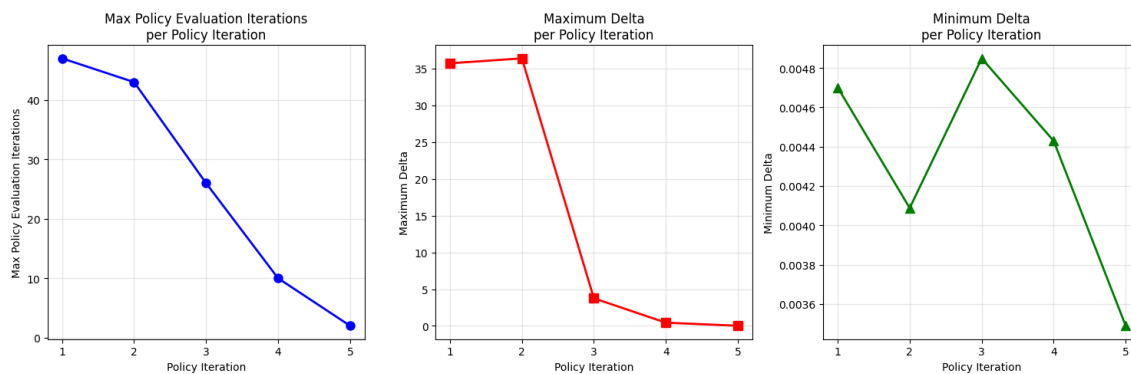
En cambio, en el escenario de 2 productos el algoritmo que más tardó fue Q Learning, con un tiempo de 3.5 minutos, mientras que PI y VI demoraron 8 y 40 segundos respectivamente. Esto tiene sentido si consideramos que la cantidad de estados crece exponencialmente con la cantidad de productos, y este crecimiento impacta más sobre los métodos exactos que sobre los aproximados, ya que Q Learning tiene una cantidad finita y prefijada de iteraciones. No necesariamente.

Value Iteration:



En estos gráficos pueden verse algunos detalles del proceso de convergencia de Value Iteration, se puede destacar que llega relativamente rápido a una solución buena, pero que después demora mucho en terminar de converger, con pequeñas ganancias en cada iteración. Teniendo un tiempo relativamente constante de ejecución dado por el loop que supone su implementación para tener la política greedy, sin embargo las ganancias en términos de convergencia decaen exponencialmente. ✓

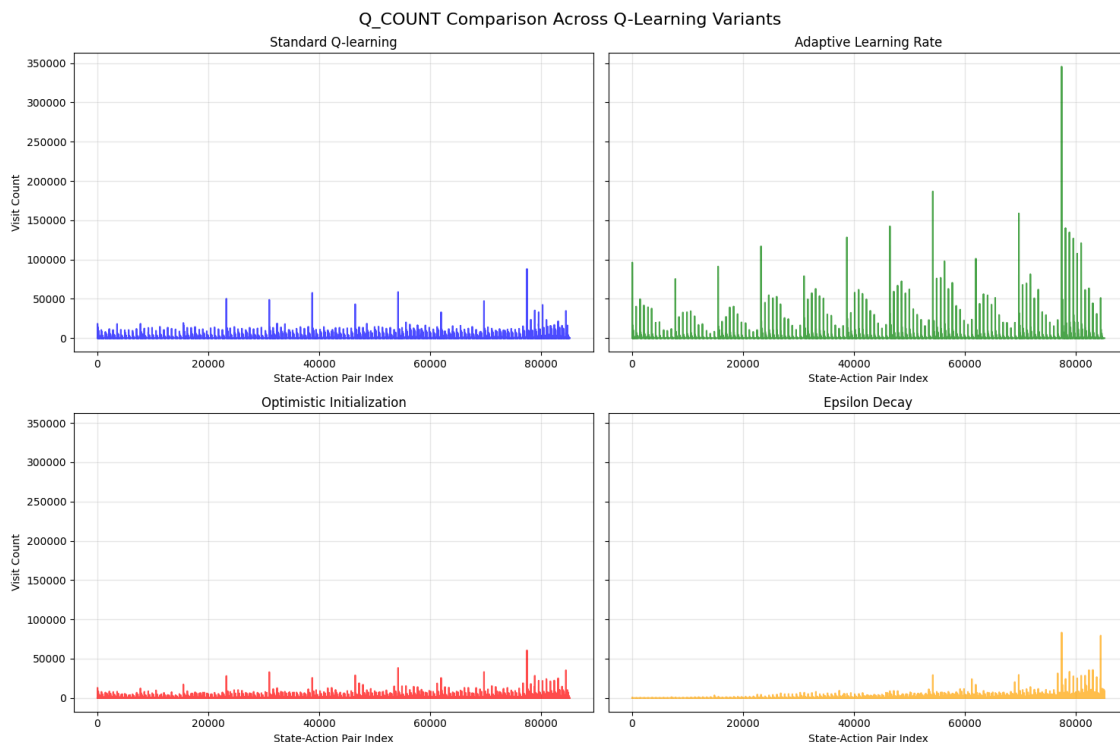
Policy Iteration:

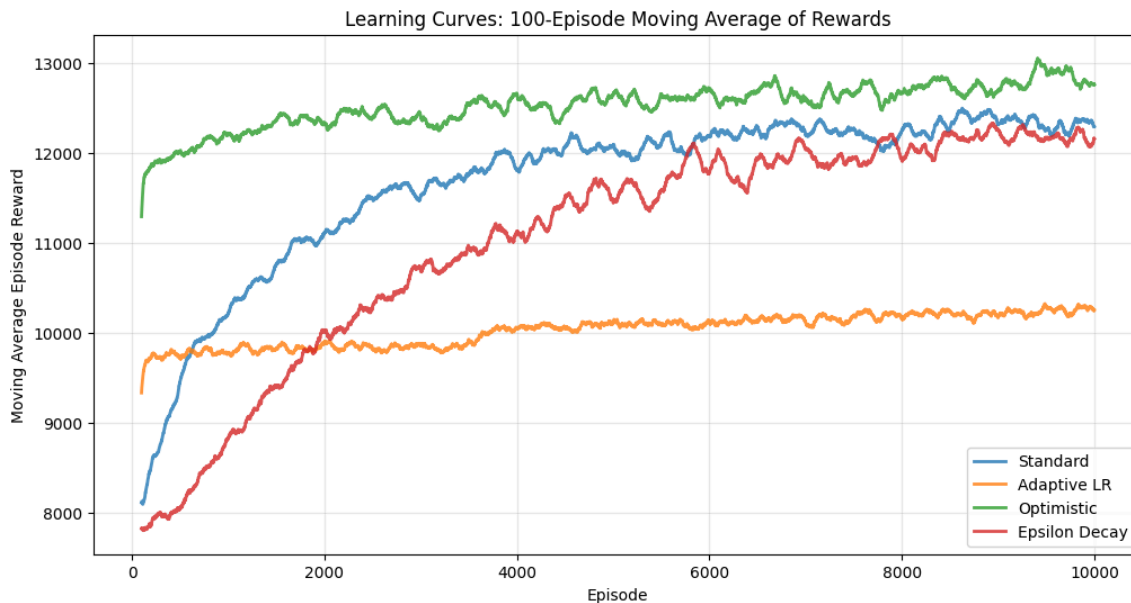


Vemos que el proceso de policy iteration muestra una disminución constante y marcada en la cantidad de iteraciones necesarias para converger al valor de la política evaluada a lo largo de los distintos procesos de evaluación de política. **Adicionalmente, podemos destacar que en cada nueva evaluación de política se utiliza información aprendida en la anterior,** lo que se ve reflejado en que el delta máximo para cada evaluación de política también presenta una tendencia decreciente.

Cómo hicieron para incorporar la información aprendida anteriormente?

Q Learning:





Excelente experimento con variantes de Q-learning!

En estos gráficos se puede ver el trade off entre exploración y explotación en las distintas variantes de Q Learning. En donde, el algoritmo estándar es la implementación vista en clase, adaptive learning rate tiene un learning rate ponderado por la cantidad de veces que se visitó ese estado acción. Finalmente, optimistic inicializa con valores positivos fomentando la exploración en las iteraciones tempranas y epsilon decay hace que el parámetro de exploración decaiga a medida de los episodios.

En particular, se destaca que con adaptive learning rate no hay demasiada exploración, lo que se ve reflejado en los picos de que tiene en ciertos pares estado-acción, mientras que hay otros que no recorre tanto. El revenue de esta variante se mantiene relativamente estable a medida que aumentan los episodios. En otras variantes como Standard o Epsilon Decay el revenue comienza en valores mucho menores y va creciendo a medida que avanzan los episodios, llegando a un revenue mayor que en el adaptativo. En estos casos se ven menos picos en pares estado-acción específicos, consistente con una mayor exploración. Por último, se destacan también los buenos resultados obtenidos con la implementación optimistic,

Sin embargo, cabe destacar que para obtener conclusiones robustas estadísticamente de la comparación de las variantes habría que repetir estos experimentos múltiples veces. ✓

Métodos exactos o aproximados:

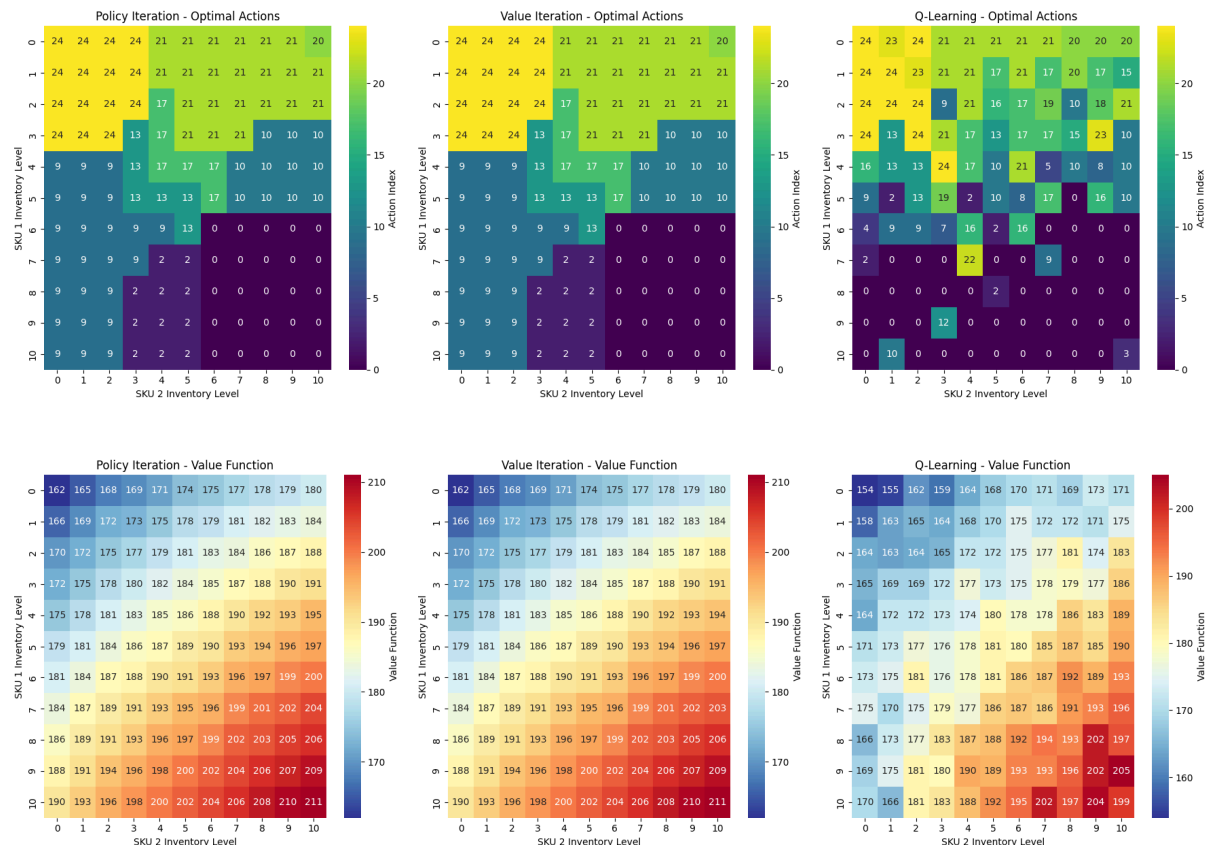
Como mencionamos anteriormente, encontramos diferencias en cuanto a cómo varían los tiempos de ejecución de los algoritmos ante nuevos productos. Mientras que en los métodos exactos el tiempo aumenta exponencialmente con la cantidad de productos, ya que depende de la cantidad de estados, en los métodos aproximados se mantiene relativamente constante al depender de la cantidad de episodios.

Además, encontramos diferencias en términos de convergencia a la política óptima que serán detallados en el punto 4.

4. Análisis de resultados

Política óptima:

Las políticas óptimas obtenidas al usar cada uno de los métodos fueron las siguientes:.



Value Function Summary Statistics:

- Policy Iteration Min: 162.30, Max: 211.17, Mean: 188.24118251246398
- Value Iteration Min: 162.29, Max: 211.16, Mean: 188.23
- Q-Learning Min: 153.74, Max: 204.71, Mean: 178.81

Es posible observar que para policy iteration y value iteration llegan a la misma política óptima, mientras que utilizando Q learning se obtiene una política distinta, no converge a la política óptima, aunque relativamente similar. También notamos que en la política óptima de las soluciones exactas se toman un número limitado de acciones, utilizando solo 9 de las 25 disponibles, mientras que en la política óptima de Q learning aparece un número mayor de acciones (por ejemplo, aparece la acción 22, la 4, la 16, la 19, la 3, etc.). **Dado que Q-learning no convergió a la política óptima, no es trivial comparar tiempos de ejecución.**

Profundizando sobre las políticas óptimas obtenidas por métodos exactos destacamos que:

- Cuando hay poco stock de ambos productos la acción elegida será la 24, que es ordenar 5 de cada uno (siendo 5 el máximo permitido por *MaxOrder*).

- Cuando hay mucho stock de ambos productos la acción óptima será la 0, es decir, ordenar 0 de cada producto.
- Cuando hay mucho stock de un producto y poco stock del otro las acciones óptimas serán la 9, la 21 o la 20, dependiendo del caso. En particular, cuando hay mucho stock del SKU1 y poco del SKU2 se toma la acción 9, que es ordenar 2 unidades del SKU1 ($MOQ = 2$) y 5 del SKU2 ($MaxOrder = 5$). La 21 y la 20 se toman en los casos en los que hay poco stock del SKU1 y mucho del SKU1, y representan ordenar 5 unidades del SKU1 y 2 o 0 unidades respectivamente del SKU2.
- En este ejemplo, la capacidad compartida del container es lo que hace que las órdenes de cada uno de los productos estén atadas entre sí. Entonces, cómo se busca llenar el container se tienen distintas acciones óptimas para combinaciones distintas de ambos productos. Esto hace que, por ejemplo, cuando el stock del SKU1 es 10 convenga pedir 2 o 0 dependiendo de cuánto sea el stock del SKU2.

Una posible explicación de la no convergencia de Q learning a la solución óptima es que no iteramos sobre los hiper parámetros del algoritmo de Q-learning como por ejemplo el learning.

Sí, y probablemente no hayan utilizado suficientes iteraciones del algoritmo en sí. Lo mismo puede verse reflejado en los valores que toma la función valor, mientras que en los métodos exactos se obtienen los mismos valores, en Q learning se obtienen valores menores, aunque cercanos, en todos los casos.

Sería interesante comparar qué tan cercanos están al óptimo.
Direcciones futuras y posibles extensiones:

En términos del modelado del problema, consideramos que es bastante representativo del problema real, sin embargo la simplificación del límite máximo independiente del producto puede tener un impacto sobre la calidad de las políticas obtenidas. Tomar una capacidad conjunta de stock y limitar un poco más el espacio de estados podría ser una extensión interesante de explorar a la hora de implementar una solución de este estilo.

Otra mejora que podría desarrollarse es la inclusión de funciones de demanda más realistas, provenientes, por ejemplo, de forecasts hechos con modelos ARMA. Para incluir esto habría que hacer algunas modificaciones, ya que la demanda pasaría a depender de demandas en periodos anteriores ($t-1$, $t-2$, $t-12$, por ejemplo), y habría que explorar de qué forma se puede incluir esta dependencia temporal.

Hay que expandir el espacio de estados.

No obstante, entendemos que el principal limitante que encontramos es la maldición de la dimensionalidad en la implementación numérica, lo cual no nos permitió explorar satisfactoriamente un conjunto de valores que puedan tener sentido en términos del problema real, en particular:

- Un número de SKUs más amplio. Actualmente modelamos un ejemplo con 2 y 3 productos, cuando en un problema real para una empresa se manejan entre decenas y miles de SKUs. Esta es una limitación importante, ya que al aumentar un solo producto la cantidad de estados crece exponencialmente, poniendo un límite computacional a las implementaciones exactas.
- Aumentar las opciones de stock disponible para darle más realismo y posibilidad de accionar al modelo. Por ejemplo, actualmente tenemos 11 opciones de stock disponible por producto, sería interesante poder aumentar esa cardinalidad.

- Trabajar con un mayor número de acciones. Actualmente decidimos sobre 5 opciones de pedido por cada producto pudiendo 0, 2, 3, 4, 5 productos. Claramente esta es una variable discreta con rangos mucho más amplios en la vida real.
- Permitir unos mayores estados de transición en la demanda. En esta implementación solo utilizamos 5 escenarios posibles de demanda para cada producto, cuando un escenario más verosímil requeriría trabajar con por lo menos 10.

Si tuvieran que elegir una modificación, cuál dirección tendría prioridad?

En particular, los primeros dos puntos hacen que la cantidad de estados crezca exponencialmente, ya que estos están dados por la cantidad de productos elevada a la cantidad de stock posibles.

Entendemos que computacionalmente, es imposible manejar en técnicas exactas este tipo de problemas, por lo que se deberían utilizar enfoques aproximados e intentar aproximar paramétricamente la función Q , en vez de aprenderla para cada estado. ✓